

Collegium Witelona Uczelnia Państwowa
Wydział Nauk Technicznych i Ekonomicznych

System identyfikacji flory

(e-zielnik)

Sprawozdanie z realizacji projektu zespołowego

Zespół projektowy

Maksym Wilk (nr albumu 43900)	Moduł serwerowy (REST API)
Adam Rudziewicz (nr albumu 43882)	Aplikacja mobilna
Szymon Rogula (nr albumu 43880)	Aplikacja webowa
Sebastian Waga (nr albumu 43894)	Aplikacja desktopowa

Przedmiot: Zespołowe metody programowania (ZMP)
Projekt zespołowy, semestr VI

Data: 8 czerwca 2026

Spis treści

1	Wstęp	3
1.1	Cel i zakres sprawozdania	3
1.2	Architektura systemu	3
1.3	Charakterystyka ilościowa projektu	3
2	Opis funkcjonalny systemu	3
2.1	Przeznaczenie systemu	3
2.2	Aktorzy systemu	4
2.3	Realizowane wymagania funkcjonalne	4
2.4	Kluczowe procesy biznesowe	6
2.4.1	Uwierzytelnianie i bezpieczeństwo konta	6
2.4.2	Identyfikacja roślin	6
2.4.3	Zielniki, zdjęcia i funkcje społecznościowe	7
2.4.4	Panel administracyjny	7
2.5	Realizacja funkcjonalności w aplikacjach klienckich	7
2.5.1	Aplikacja mobilna	7
2.5.2	Aplikacja webowa	7
2.5.3	Aplikacja desktopowa	8
3	Opis technologiczny	9
3.1	Moduł serwerowy (REST API)	9
3.1.1	Stos technologiczny	9
3.1.2	Architektura aplikacji	9
3.1.3	Model danych	10
3.1.4	Bezpieczeństwo	11
3.1.5	Integracje zewnętrzne	11
3.1.6	Przechowywanie zdjęć	12
3.1.7	Interfejs REST	12
3.1.8	Testy i jakość kodu	14
3.1.9	Integracja ciągła (CI)	14
3.2	Aplikacja mobilna (Flutter)	14
3.2.1	Stos i architektura	14
3.2.2	Zarządzanie stanem i warstwa sieciowa	15
3.2.3	Pamięć lokalna i wstrzykiwanie zależności	15
3.2.4	Testy i jakość kodu	15
3.3	Aplikacja webowa (React / TypeScript)	15
3.3.1	Stos technologiczny	15
3.3.2	Architektura aplikacji	15
3.3.3	Wzorce projektowe	16
3.3.4	Autoryzacja i internacjonalizacja	17
3.4	Aplikacja desktopowa (WPF / .NET)	17
3.4.1	Stos i architektura	17
3.4.2	Komunikacja z API i bezpieczeństwo	17
3.4.3	Tryb offline i synchronizacja	18
3.4.4	Wielojęzyczność, wzorce i testy	18

4	Podział obowiązków i odpowiedzialności w zespole	18
5	Instrukcja uruchomienia systemu	19
5.1	Moduł serwerowy (REST API)	19
5.1.1	Wymagania wstępne	19
5.1.2	Konfiguracja zmiennych środowiskowych	19
5.1.3	Uruchomienie lokalne	20
5.1.4	Uruchomienie kontenerowe (Docker Compose)	21
5.1.5	Uruchomienie zdalne (Railway)	21
5.2	Aplikacja mobilna (Flutter)	21
5.2.1	Uruchomienie deweloperskie	21
5.2.2	Budowa pakietu produkcyjnego (APK)	22
5.3	Aplikacja webowa (React)	22
5.3.1	Wymagania wstępne	22
5.3.2	Uruchomienie lokalne	22
5.3.3	Uruchomienie produkcyjne (Vercel)	23
5.4	Aplikacja desktopowa (WPF)	23
5.4.1	Wymagania wstępne	23
5.4.2	Uruchomienie i budowa	23
6	Wnioski projektowe	23
6.1	Moduł serwerowy (Maksym Wilk)	23
6.2	Aplikacja mobilna (Adam Rudziewicz)	24
6.3	Aplikacja webowa (Szymon Rogula)	24
6.4	Aplikacja desktopowa (Sebastian Waga)	24
7	Podsumowanie	25

1 Wstęp

Niniejsze sprawozdanie dokumentuje projekt zespołowy **e-zielnik**, czyli system identyfikacji flory umożliwiający prowadzenie cyfrowego zielnika roślin. System pozwala fotografować rośliny, automatycznie rozpoznawać ich gatunek za pomocą zewnętrznego serwisu PlantNet, gromadzić je w nazwanych zielnikach, opisywać, udostępniać publicznie lub znajomym oraz otrzymywać powiadomienia. Część administracyjna udostępnia statystyki i narzędzia moderacyjne.

1.1 Cel i zakres sprawozdania

Celem dokumentu jest całościowe przedstawienie zrealizowanego systemu: jego funkcji, przyjętych rozwiązań technologicznych, podziału prac w zespole, sposobu uruchomienia poszczególnych modułów oraz wniosków końcowych każdego z autorów. Sprawozdanie obejmuje wszystkie cztery moduły projektu, traktowane jako jedna spójna całość.

1.2 Architektura systemu

System ma architekturę klient-serwer. Centralnym elementem jest **REST API**, z którego korzystają trzy niezależne aplikacje klienckie:

- **Moduł serwerowy (REST API)** – napisany w technologiach Java oraz Spring Boot, z bazą danych PostgreSQL; realizuje całość logiki biznesowej i przechowuje dane.
- **Aplikacja mobilna** – wieloplatformowy klient zbudowany w technologii Flutter (Dart), przeznaczony dla użytkowników końcowych w terenie.
- **Aplikacja webowa** – jednostronicowa aplikacja (SPA) w technologii React oraz TypeScript, dostępna z poziomu przeglądarki.
- **Aplikacja desktopowa** – natywny panel administracyjny dla komputerów stacjonarnych, zbudowany w technologii WPF (.NET 8).

Wszystkie aplikacje klienckie komunikują się z serwerem w formacie JSON poprzez protokół HTTP. Stan sesji jest bezstanowy – uwierzytelnianie odbywa się tokenem JWT.

1.3 Charakterystyka ilościowa projektu

Najważniejsze wielkości charakteryzujące poszczególne moduły zestawia tabela 1.

2 Opis funkcjonalny systemu

2.1 Przeznaczenie systemu

API realizuje logikę biznesową cyfrowego zielnika. Odpowiada za rejestrację i uwierzytelnianie użytkowników, zarządzanie zielnikami oraz roślinami, identyfikację gatunków na podstawie zdjęć, przechowywanie i serwowanie plików graficznych, funkcje społecznościowe (znajomi), powiadomienia (w tym powiadomienia push) oraz panel administracyjny. Aplikacje klienckie udostępniają te funkcje użytkownikom w sposób dostosowany do swojej platformy.

Tabela 1: Charakterystyka ilościowa modułów projektu

Moduł	Wybrane wielkości
REST API	ok. 6 600 linii kodu produkcyjnego Java w 87 plikach, ok. 3 900 linii kodu testów w 22 plikach, 10 modułów dziedzinowych, 9 encji, 8 kontrolerów REST, próg pokrycia testami 80 % (JaCoCo); 77 rewizji w okresie marzec–czerwiec 2026
Aplikacja mobilna	architektura warstwowa (Clean Architecture), wzorzec BLoC, 33 testy jednostkowe, modele niemutowalne (Freezed)
Aplikacja webowa	React 18 / TypeScript 5, architektura SPA, osiem świadomie zastosowanych wzorców projektowych, autorska warstwa i18n
Aplikacja desktopowa	WPF / .NET 8, architektura MVVM, lokalna baza SQLite, tryb offline z kolejką synchronizacji, testy jednostkowe xUnit

2.2 Aktorzy systemu

W systemie występują trzy role:

Gość Użytkownik niezalogowany. Może się zarejestrować, zalogować, zresetować hasło oraz przeglądać publiczne zielniki wraz z ich roślinami i zdjęciami.

Użytkownik

Konto zalogowane i ze zweryfikowanym adresem e-mail. Zarządza własnymi zielnikami i roślinami, dodaje rośliny przez identyfikację ze zdjęcia, nawiązuje znajomości, otrzymuje powiadomienia i może włączyć uwierzytelnianie dwuskładnikowe.

Administrator

Konto z flagą `is_admin`. Posiada wgląd w statystyki systemu oraz narzędzia moderacyjne (blokowanie kont, ostrzeżenia, usuwanie treści).

Rola administratora nie jest osobną encją, lecz wynika z pola logicznego `is_admin` w tabeli użytkowników. Status konta wyznaczają dwie flagi: `is_active` (konto aktywne) oraz `is_verified` (zweryfikowany e-mail), egzekwowane globalnie przez `AccountStatusInterceptor`.

2.3 Realizowane wymagania funkcjonalne

Poniższa tabela zestawia wymagania funkcjonalne projektu (oznaczenia OPZ FLY-xx) wraz ze sposobem ich realizacji po stronie API.

Nr	Funkcjonalność	Realizacja w API
FLY-01	Pierwszy administrator dodany do systemu	Konto pierwszego administratora jest osadzone bezpośrednio w bazie danych; kolejni administratorzy przez <code>/users/{userId}/make-admin</code>
FLY-02	Logowanie administratora	POST <code>/users/login</code>

ciąg dalszy na następnej stronie

Nr	Funkcjonalność	Realizacja w API	
FLY-03	Podgląd wszystkich zielników	GET /stats/herbaria, /stats/herbaria/{id}	GET
FLY-04	Usuwanie zielników użytkowników	Usunięcie zielnika użytkownika (panel administratora)	
FLY-05	Usuwanie pojedynczych roślin i zdjęć	DELETE .../plants/{plantId} .../photos/{photoId}	oraz
FLY-06	Usuwanie i blokowanie użytkowników	PATCH /users/{id}/ban, /unban, DELETE /users/{id}	
FLY-07	Ostrzeżenia i powiadomienia push	POST /users/{id}/warning (e-mail + powiadomienie + FCM)	
FLY-08	Statystyki systemowe	GET /stats/overview, /stats/users	
FLY-09	Przeglądanie publicznych zielników przez gościa	GET /herbaria/public, /herbaria/{id}/plants	GET
FLY-10	Rejestracja	POST /users/register	
FLY-11	Logowanie	POST /users/login	
FLY-12	Reset hasła	POST /users/forgot-password, /users/reset-password	
FLY-13	Dodawanie zielników	POST /herbaria	
FLY-14	Dodawanie zdjęć i wykrywanie rośliny	POST /herbaria/{id}/plants/add /confirm	+
FLY-15	Opisy roślin i zdjęć	PATCH .../plants/{id}, .../photos/{id}	PATCH
FLY-16	Lista własnych zielników i roślin	GET /herbaria/me, GET .../plants	
FLY-17	Ustawianie zielnika jako publicznego	PATCH /herbaria/{id} (pole isPublic)	
FLY-18	Dodawanie znajomych	POST /friends/request, /accept	
FLY-19	Widok zielników znajomych i publicznych	GET /herbaria/user/{userId}	(kontrola areFriends)
FLY-20	Przeglądanie i porównywanie roślin znajomych	GET .../plants, .../plants/{id}	
FLY-21	Wylogowanie	POST /users/logout (unieważnienie refresh tokenu)	
FLY-22	Obsługa języka polskiego i angielskiego	Komunikaty API w języku angielskim; warstwa językowa interfejsu realizowana po stronie aplikacji klienckich	

2.4 Kluczowe procesy biznesowe

2.4.1 Uwierzytelnianie i bezpieczeństwo konta

Rejestracja (POST /users/register) waliduje siłę hasła (minimum 8 znaków, co najmniej jedna wielka litera, jedna cyfra oraz jeden znak specjalny), sprawdza unikalność adresu e-mail i nazwy użytkownika, zapisuje hash hasła (BCrypt) i wysyła wiadomość weryfikacyjną przez usługę Brevo. Adres e-mail jest normalizowany (przycięcie i zamiana na małe litery).

Logowanie (POST /users/login) akceptuje login w postaci adresu e-mail lub nazwy użytkownika. Po pomyślnym uwierzytelnieniu klient otrzymuje parę tokenów: krótko żyjący token dostępu JWT (domyślnie 2 godziny, algorytm HS512) oraz długo żyjący, nieprzezroczysty refresh token (domyślnie 30 dni). Odświeżanie pary tokenów (POST /users/refresh) odbywa się z rotacją: poprzedni refresh token jest kasowany, a wydawany jest nowy. System wykrywa ponowne użycie unieważnionego refresh tokenu i w reakcji unieważnia wszystkie sesje danego użytkownika.

Opcjonalne uwierzytelnianie dwuskładnikowe (2FA) wykorzystuje 6-cyfrowy kod wysyłany pocztą elektroniczną. Gdy konto ma włączone 2FA, logowanie zwraca token przejściowy (pre_auth, ważny 5 minut), a pełen dostęp uzyskuje się dopiero po potwierdzeniu kodu (POST /users/verify-2fa). Reset hasła wykorzystuje token JWT zawierający tzw. wersję hasła (HMAC-SHA256 obliczony nad hashem hasła). Dzięki temu link resetujący automatycznie traci ważność po zmianie hasła, bez przechowywania stanu w bazie.

2.4.2 Identyfikacja roślin

Najważniejszą funkcją domenową jest dodawanie roślin przez identyfikację ze zdjęcia. Proces jest dwuetapowy:

1. Użytkownik wysyła zdjęcie (POST /herbaria/{id}/plants/add, multipart/form-data). API przekazuje obraz do serwisu PlantNet i analizuje wynik. Możliwe są trzy scenariusze:
 - **resolved** – rozpoznano gatunek, a w zielniku istnieje już dopasowana roślina (po identyfikatorze gatunku GBIF lub nazwie gatunkowej); zdjęcie zostaje od razu dołączone do istniejącej rośliny;
 - **recognized** – rozpoznano gatunek, lecz istnieją tylko częściowo pasujące rośliny; system zwraca listę rekomendacji i wymaga decyzji użytkownika;
 - **unrecognized** – gatunek nierozpoznany; wymagane jest potwierdzenie.

W dwóch ostatnich przypadkach zdjęcie jest tymczasowo zapisywane w katalogu **pending**, a metadane identyfikacji przechowywane są w pamięci.

2. Użytkownik potwierdza decyzję (POST /herbaria/{id}/plants/confirm), wybierając przypisanie do istniejącej rośliny (**existing**) lub utworzenie nowej (**new**). Plik jest przenoszony z katalogu tymczasowego do stałego, a rekord rośliny tworzony lub aktualizowany.

Wpisy oczekujące wygasają po godzinie i są usuwane przez zadanie cykliczne uruchamiane co 5 minut. Roślina przechowuje dane taksonomiczne pobrane z PlantNet: wykryty gatunek, identyfikator GBIF, rodzinę, rodzaj oraz nazwy zwyczajowe.

2.4.3 Zielniki, zdjęcia i funkcje społecznościowe

Zielnik jest nazwanym pojemnikiem na rośliny, domyślnie prywatnym. Obowiązują limity: maksymalnie 50 zielników na użytkownika, 200 roślin na zielnik oraz 50 zdjęć na roślinę. Nazwa zielnika musi być unikalna w obrębie jednego użytkownika (wymuszone również ograniczeniem bazodanowym). Usunięcie ostatniego zdjęcia rośliny automatycznie usuwa samą roślinę (roślina nie istnieje bez zdjęć).

Funkcje społecznościowe opierają się na encji znajomości o statusie **PENDING** lub **ACCEPTED**. Zaakceptowana znajomość nadaje obu stronom dostęp do prywatnych zielników, roślin i zdjęć drugiej osoby. Zaproszenia oraz ich akceptacja generują powiadomienia w aplikacji oraz, jeśli skonfigurowano Firebase, powiadomienia push na urządzenia mobilne.

2.4.4 Panel administracyjny

Administrator dysponuje statystykami zbiorczymi (liczby użytkowników, zielników, roślin i znajomości), może przeglądać szczegóły kont i ich treści, blokować i odblokowywać konta, nadawać i odbierać rolę administratora, wysyłać ostrzeżenia (równoległe e-mailem i powiadomieniem push) oraz usuwać treści użytkowników. Usunięcie konta jest realizowane jako anonimizacja (tzw. miękkie usunięcie): konto jest dezaktywowane, a adres e-mail i nazwa użytkownika nadpisywane wzorcami `deleted-...`, co zachowuje integralność powiązań w bazie.

2.5 Realizacja funkcjonalności w aplikacjach klienckich

2.5.1 Aplikacja mobilna

Aplikacja mobilna dostarcza użytkownikom końcowym interaktywnego interfejsu do rozpoznawania roślin w terenie. Główne funkcje obejmują:

- **Identyfikacja wspierana sztuczną inteligencją** – wykonywanie zdjęć wbudowanym aparatem i przysyłanie ich na serwer w celu analizy modelem uczenia maszynowego; wyniki zwracane są wraz z procentowym stopniem pewności.
- **Herbarium (zielnik offline)** – tworzenie cyfrowej kolekcji odnalezionych roślin z możliwością przeglądania bez dostępu do internetu dzięki wbudowanej, szybkiej bazie lokalnej.
- **Zarządzanie bezpieczeństwem i profilem** – rejestracja, uwierzytelnianie tokenami JWT z automatycznym odświeżaniem sesji oraz opcjonalne zabezpieczenie biometryczne (odcisk palca lub kod PIN urządzenia).
- **Powiadomienia push** – odbieranie powiadomień asynchronicznych dzięki integracji z *Firebase Cloud Messaging* (FCM).
- **Wielojęzyczność** – dynamiczna zmiana języka interfejsu (polski lub angielski) bez restartu aplikacji.

2.5.2 Aplikacja webowa

Aplikacja webowa stanowi przeglądarkowy interfejs systemu. Zaimplementowane funkcje to:

- **Uwierzytelnianie użytkowników** – rejestracja z walidacją hasła w czasie rzeczywistym (siła hasła, wymagania bezpieczeństwa), logowanie z obsługą tokenów JWT, automatyczne odświeżanie sesji, wylogowanie z czyszczeniem stanu oraz automatyczne wylogowanie po 15 minutach bezczynności.
- **Zarządzanie zielnikami** – przeglądanie własnych zielników, widok roślin wraz ze zdjęciami i opisami botanicznymi (gatunek, rodzina, rodzaj, nazwy zwyczajowe).
- **Przeglądanie publicznych zielników** – dostęp do kolekcji oznaczonych jako publiczne bez konieczności logowania.
- **System znajomych** – wyszukiwanie użytkowników po nazwie, wysyłanie i obsługa zaproszeń oraz przeglądanie zielników znajomych.
- **Porównywanie roślin** – panel boczny umożliwiający porównanie rośliny ze zbioru znajomego lub publicznego z własną rośliną.
- **System powiadomień** – dzwonek powiadomień w pasku nawigacji z odpytywaniem co 30 sekund, oznaczanie jako przeczytane oraz usuwanie powiadomień.
- **Internacjonalizacja (i18n)** – pełne tłumaczenie interfejsu na język polski i angielski, przełączanie w czasie rzeczywistym bez przeładowania strony, tłumaczenie komunikatów walidacji.
- **Responsywność** – menu typu hamburger na urządzeniach mobilnych i dostosowanie układu do różnych rozdzielczości.
- **Odzyskiwanie hasła** – formularz resetowania hasła przez e-mail.

2.5.3 Aplikacja desktopowa

Aplikacja desktopowa jest narzędziem administracyjnym uruchamianym na komputerach stacjonarnych. Umożliwia administratorowi między innymi:

- logowanie do systemu oraz zarządzanie użytkownikami,
- nadawanie i odbieranie uprawnień administratora,
- blokowanie i odblokowywanie użytkowników oraz wysyłanie ostrzeżeń,
- usuwanie kont użytkowników,
- zarządzanie roślinami i przeglądanie statystyk systemu,
- obsługę powiadomień,
- pracę w trybie offline z lokalną bazą danych.

Tabela 3: Stos technologiczny modułu serwerowego

Technologia	Wersja	Zastosowanie
Java	21	Język programowania (LTS)
Spring Boot	4.0.6	Szkielet aplikacji, autokonfiguracja
Spring Web (MVC)	–	Warstwa REST
Spring Data JPA / Hibernate	–	Mapowanie obiektowo-relacyjne (ORM)
Spring Security + OAuth2 RS	–	Uwierzytelnianie i autoryzacja (JWT)
Bean Validation	–	Walidacja danych wejściowych
PostgreSQL	–	Relacyjna baza danych
JWT (jjwt)	0.13.0	Generowanie i podpisywanie tokenów JWT
springdoc-openapi	3.0.2	Dokumentacja OpenAPI / Swagger UI
Bucket4j	8.17.0	Ograniczanie liczby żądań (token bucket)
Caffeine	3.2.4	Pamięć podręczna dla rate limiting
TwelveMonkeys imageio-webp	3.12.0	Dekodowanie obrazów WebP
Firebase Admin SDK	9.9.0	Powiadomienia push (FCM)
Apache HttpClient 5	–	Klient HTTP
Testcontainers	1.21.4	Testy integracyjne na realnym PostgreSQL
JaCoCo	0.8.14	Pomiar pokrycia testami
Maven (wrapper)	–	Budowanie i zarządzanie zależnościami
Docker	–	Konteneryzacja (build wieloetapowy)

3 Opis technologiczny

3.1 Moduł serwerowy (REST API)

3.1.1 Stos technologiczny

3.1.2 Architektura aplikacji

Aplikacja ma klasyczną budowę warstwową stosowaną w aplikacjach Spring Boot:

Kontrolery (***Controller**)

przyjmują żądania HTTP, odczytują tożsamość z tokenu JWT i delegują logikę do serwisów.

Serwisy (***Service**)

zawierają logikę biznesową, walidacje i transakcje.

Repozytoria (***Repository**)

interfejsy Spring Data JPA realizujące dostęp do bazy (zapytania pochodne i adnotacja `@Query`).

Encje (**@Entity**)

obiekty mapowane na tabele bazy danych.

Obiekty DTO (***Request** / ***Response**)

modele danych wejścia i wyjścia, odseparowane od encji.

Kod jest podzielony na pakiety dziedzinowe (jeden pakiet na obszar funkcjonalny), co ułatwia orientację w projekcie:

```

com.ezielnik.api
|-- admin                panel administracyjny
| |-- content_management moderacja tresci (zielniki, rosliny, zdjecia)
| |-- user_management    moderacja kont uzytkownikow
|-- auth                 uwierzytelnianie
| |-- refresh_token      rotacja tokenow odswiezajacych
| |-- two_factor_auth    uwierzytelnianie dwuskladnikowe (2FA)
|-- config               konfiguracja (bezpieczenstwo, rate limit, bledy)
|-- fcm                  powiadomienia push (Firebase Cloud Messaging)
|-- friend               znajomi
|-- herbarium            zielniki
|-- notification         powiadomienia w aplikacji
|-- photo                przechowywanie i serwowanie zdjec
|-- plant               rosliny i identyfikacja (PlantNet)
|-- user                konta uzytkownikow (oraz endpointy auth)
'-- ApiApplication       klasa startowa

```

Listing 1: Struktura pakietow modulu API

Punkty końcowe uwierzytelniania nie mają osobnego kontrolera, lecz znajdują się w UserController pod prefiksem /users.

3.1.3 Model danych

Baza danych zawiera 9 encji. Wszystkie klucze główne są typu UUID, generowane po stronie aplikacji w metodzie @PrePersist. Znaczniki czasu wykorzystują typ OffsetDateTime. Centralnym węzłem modelu jest encja User, do której odwołują się wszystkie pozostałe encje.

Tabela 4: Encje modelu danych modulu API

Encja	Tabela	Najważniejsze pola i powiązania
User	users	e-mail (unikalny), nazwa (unikalna), hash hasła, flagi is_active, is_verified, is_admin, email_two_factor_enabled
Herbarium	herbaria	nazwa, opis, is_public; FK user_id; unikalność pary (user_id, name)
Plant	plants	nazwa, detected_species, species_id, rodzina, rodzaj, nazwy zwyczajowe; FK herbarium_id
PlantPhoto	plant_photos	url, opis, confidence; FK plant_id
Friendship	friendships	status (PENDING/ACCEPTED); dwa FK do User: requester_id, addressee_id
Notification	notifications	title, message, is_read; FK user_id
RefreshToken	refresh_tokens	token_hash (unikalny), expires_at; FK user_id
TwoFactorCode	two_factor_codes	code_hash, expires_at, used; FK user_id
DeviceToken	device_tokens	token (unikalny, FCM); FK user_id

Główną hierarchię stanowi trójpoziomowa kompozycja zielnika (User 1:N Herbarium 1:N Plant 1:N PlantPhoto). Encja Friendship modeluje relację samozwrotną na User (dwa odrębne odwołania). Schematyczny model powiązań przedstawia poniższy diagram:

```

User (users)
|
|-- 1:N --> Herbarium (herbaria)

```

```

|
|      +--- 1:N --> Plant (plants)
|      |
|      |      +--- 1:N --> PlantPhoto (plant_photos)
|
|
|-- 1:N --> Notification (notifications)
|-- 1:N --> RefreshToken (refresh_tokens)
|-- 1:N --> DeviceToken (device_tokens)
|-- 1:N --> TwoFactorCode (two_factor_codes)
|
|-- 1:N (requester_id / addressee_id) --> Friendship (friendships)

```

Listing 2: Powiązania między encjami (uproszczony diagram ER)

Schemat bazy jest tworzony automatycznie przez Hibernate (`spring.jpa.hibernate.ddl-auto=update`).

3.1.4 Bezpieczeństwo

Bezpieczeństwo jest realizowane na kilku poziomach:

- Hasła i kody są haszowane algorytmem BCrypt (`BCryptPasswordEncoder`); dotyczy to również jednorazowych kodów 2FA.
- Tokeny JWT są podpisywane algorytmem symetrycznym HS512 (klucz z konfiguracji `JWT_SECRET`). Dekodowanie realizuje `NimbusJwtDecoder`. Typy tokenów (dostępu, przejściowy `pre_auth`, weryfikacji e-mail, resetu hasła) są rozróżniane przez deklarację `purpose`, co zapobiega użyciu tokenu w niewłaściwym kontekście.
- Refresh tokeny są przechowywane wyłącznie jako skrót SHA-256 (nigdy w postaci jawnej), z rotacją oraz wykrywaniem ponownego użycia.
- Konfiguracja Spring Security jest bezstanowa (`SessionCreationPolicy.STATELESS`), z wyłączonym CSRF, uwierzytelnianiem Basic i logowaniem formularzowym. Niestandardowy `BearerTokenResolver` pomija parsowanie tokenu dla ściśle wyliczonej listy ścieżek publicznych.
- Kontrola stanu konta (`AccountStatusInterceptor`) blokuje dostęp kontom nieaktywnym i niezweryfikowanym oraz egzekwuje, że token `pre_auth` działa wyłącznie na ścieżkach 2FA.
- Ograniczanie liczby żądań (`RateLimitFilter`) wykorzystuje `Bucket4j` z pamięcią podręczną `Caffeine`, działając per adres IP (domyślnie 30 żądań na minutę). Po przekroczeniu limitu zwracany jest kod 429 `Too Many Requests` z nagłówkiem `Retry-After`.
- Kontrola dostępu na poziomie obiektu sprawdza własność zasobu (zielnika, rośliny, zdjęcia, powiadomienia), a przy odczycie również relację znajomości oraz status publiczny. Operacje na plikach są zabezpieczone przed atakiem typu path traversal.

3.1.5 Integracje zewnętrzne

PlantNet

serwis identyfikacji roślin (`my-api.plantnet.org/v2/identify/all`). API wysyła zdjęcie metodą POST (multipart) z parametrami `nb-results=1` oraz `lang=en` i przetwarza najlepszy wynik. Wszelkie błędy wywołania są obsługiwane bezpiecznie (degradacja do ścieżki „gatunek nierozpoznany”).

Brevo

transakcyjna poczta elektroniczna (weryfikacja e-mail, reset hasła, kody 2FA, ostrzeżenia administracyjne), wywoływana przez `RestClient`.

Firebase Cloud Messaging

powiadomienia push wysyłane jako wiadomość multicast do wszystkich tokenów urządzeń użytkownika. Inicjalizacja Firebase jest opcjonalna (zmienna `FIREBASE_SERVICE_ACCOUNT_JSON`); przy jej braku powiadomienia w aplikacji nadal działają, a push jest cicho wyłączany. Nieaktualne tokeny urządzeń są automatycznie usuwane.

3.1.6 Przechowywanie zdjęć

Zdjęcia są obsługiwane przez `PhotoStorageService`. Każdy przesłany obraz (także w formacie WebP, dekodowanym dzięki bibliotece `TwelveMonkeys`) jest konwertowany do formatu JPEG z jakością 0,85 i zapisywany pod losową nazwą `UUID.jpeg`. Pliki przechowywane są w katalogu wskazanym przez `PHOTO_STORAGE_PATH` (z podkatalogiem `pending` na pliki oczekujące). Serwowanie plików (`GET /photos/{filename}`) ustawia długą pamięć podręczną (`Cache-Control: public, max-age=31536000, immutable`), co jest bezpieczne dzięki niezmiennym nazwom plików.

3.1.7 Interfejs REST

Poniższa tabela zestawia najważniejsze punkty końcowe API w podziale na kontrolery.

Metoda	Ścieżka	Dostęp
UserController (uwierzytelnianie i konto), prefiks <code>/users</code>		
POST	<code>/users/register</code>	publiczny
POST	<code>/users/login</code>	publiczny
POST	<code>/users/verify-2fa</code>	token <code>pre_auth</code>
POST	<code>/users/2fa/email/enable</code>	zalogowany
POST	<code>/users/2fa/disable</code>	zalogowany
POST	<code>/users/2fa/send-email-code</code>	token <code>pre_auth</code>
GET	<code>/users/me</code>	zalogowany
DELETE	<code>/users/me</code>	zalogowany
GET	<code>/users/verify</code>	publiczny
POST	<code>/users/resend-verification</code>	publiczny
POST	<code>/users/forgot-password</code>	publiczny
GET	<code>/users/reset-password</code>	publiczny (formularz)
POST	<code>/users/reset-password</code>	publiczny
POST	<code>/users/refresh</code>	publiczny (refresh token)
POST	<code>/users/logout</code>	publiczny (refresh token)
POST	<code>/users/me/fcm-token</code>	zalogowany
DELETE	<code>/users/me/fcm-token</code>	zalogowany
HerbariumController, prefiks <code>/herbaria</code>		
POST	<code>/herbaria</code>	zalogowany
GET	<code>/herbaria/me</code>	zalogowany
GET	<code>/herbaria/public</code>	publiczny

ciąg dalszy na następnej stronie

Metoda	Ścieżka	Dostęp
GET	/herbaria/user/{userId}	zalogowany
GET	/herbaria/{id}	właściciel / publiczny / znajomy / admin
PATCH	/herbaria/{id}	właściciel
DELETE	/herbaria/{id}	właściciel
PlantController, prefiks /herbaria/{herbariumId}/plants		
POST	/.../plants/add	właściciel
POST	/.../plants/confirm	właściciel
GET	/.../plants	publiczny / znajomy / właściciel
GET	/.../plants/{plantId}	publiczny / znajomy / właściciel
PATCH	/.../plants/{plantId}	właściciel
DELETE	/.../plants/{plantId}	właściciel
GET	/.../plants/{plantId}/photos/{photoId}	publiczny / znajomy / właściciel
PATCH	/.../plants/{plantId}/photos/{photoId}	właściciel
DELETE	/.../plants/{plantId}/photos/{photoId}	właściciel
POST	/.../plants/{plantId}/photos/{photoId}/ move	właściciel
PhotoController		
GET	/photos/{filename}	publiczny / znajomy / właściciel
FriendshipController, prefiks /friends		
POST	/friends/request	zalogowany
POST	/friends/{id}/accept	adresat
DELETE	/friends/{id}	strona relacji
GET	/friends	zalogowany
GET	/friends/requests	zalogowany
GET	/friends/requests/sent	zalogowany
NotificationController, prefiks /notifications		
GET	/notifications	zalogowany
GET	/notifications/unread	zalogowany
PATCH	/notifications/{id}/read	właściciel
PATCH	/notifications/read-all	zalogowany
Kontrolery administracyjne (/stats, /users)		
GET	/stats/overview	administrator
GET	/stats/users	administrator
GET	/stats/users/{userId}	administrator
GET	/stats/users/{userId}/friends	administrator
GET	/stats/herbaria	administrator
GET	/stats/herbaria/{herbariumId}	administrator
PATCH	/users/{userId}/ban	administrator
PATCH	/users/{userId}/unban	administrator

ciąg dalszy na następnej stronie

Metoda	Ścieżka	Dostęp
PATCH	/users/{userId}/make-admin	administrator
PATCH	/users/{userId}/remove-admin	administrator
POST	/users/{userId}/warning	administrator
DELETE	/users/{userId}	administrator
DELETE	/users/{userId}/herbaria/{herbariumId}	administrator
DELETE	/users/{userId}/.../plants/{plantId}	administrator
DELETE	/users/{userId}/.../photos/{photoId}	administrator

Pełna, interaktywna dokumentacja jest generowana automatycznie (springdoc-openapi) i dostępna pod adresem `/swagger-ui.html` oraz `/v3/api-docs`.

3.1.8 Testy i jakość kodu

Strategia testowania jest dwupoziomowa:

- **Testy integracyjne** (sufiks `IT`, 14 plików) uruchamiają pełny kontekst Spring Boot na losowym porcie i korzystają z prawdziwej bazy PostgreSQL udostępnianej przez Testcontainers. Klasa bazowa `IntegrationTestBase` czyści bazę przed każdym testem oraz dostarcza metody pomocnicze (rejestracja, logowanie, nagłówki autoryzacji). Usługi zewnętrzne (Brevo, PlantNet, magazyn plików) są podmieniane na atrapy (`@MockitoBean`).
- **Testy jednostkowe** (sufiks `Test`, 4 pliki) działają bez kontekstu Springa, na czystym JUnit 5 i Mockito (m.in. logika JWT, rotacja refresh tokenów, konwersja zdjęć, mechanizm roślin oczekujących).

Pokrycie testami jest mierzone wtyczką JaCoCo z progiem 80% instrukcji (z wyłączeniem pakietu konfiguracyjnego oraz klas DTO). Zestaw testów obejmuje między innymi kontrolę dostępu, ograniczanie liczby żądań, walidację danych, odporność na awarię poczty oraz limit rozmiaru przesłanego pliku.

3.1.9 Integracja ciągła (CI)

Repozytorium korzysta z GitHub Actions (plik `.github/workflows/ci.yml`). Przy każdym push oraz pull request na gałąź `master` uruchamiany jest job, który konfiguruje JDK 21, wykonuje `./mvnw verify -B` (kompilacja, testy oraz weryfikacja progu pokrycia), a na końcu publikuje raport JaCoCo jako artefakt. W środowisku CI mechanizm Ryuk Testcontainers jest wyłączony.

3.2 Aplikacja mobilna (Flutter)

3.2.1 Stos i architektura

Aplikacja mobilna to wieloplatformowy projekt zrealizowany w frameworku **Flutter** przy użyciu języka **Dart**. Zastosowano wzorzec **Clean Architecture** połączony ze wzorcem **Repository Pattern**, z hermetycznym podziałem na warstwę *domeny* (czysta logika biznesowa, abstrakcje), warstwę *danych* (konkretne implementacje) oraz warstwę *prezentacji* (UI). Architektura ściśle przestrzega reguł **SOLID**, **DRY** oraz **KISS**.

3.2.2 Zarządzanie stanem i warstwa sieciowa

Komunikacja pomiędzy interfejsem graficznym a logiką biznesową opiera się na re-aktywnym wzorcu *Business Logic Component (BLoC)*. Interfejs użytkownika słucha emitowanych strumieni danych (*Stream*), reagując wyłącznie na jawnie zdefiniowane stany.

Do obsługi protokołu HTTP wykorzystano bibliotekę *Dio*. Zaimplementowano w niej wzorzec *Interceptor*, który automatycznie dokleja i odświeża tokeny JWT, zdejmując z programisty obowiązek dopisywania nagłówków bezpieczeństwa do każdego żądania. Do obsługi błędów napisano globalny, scentralizowany mapper wyjątków.

3.2.3 Pamięć lokalna i wstrzykiwanie zależności

Jako lokalną bazę danych zastosowano system *Hive* – wydajną bazę NoSQL o strukturze klucz–wartość, gwarantującą błyskawiczne wczytywanie zielnika z pominięciem opóźnień sieciowych. Wrażliwe dane, takie jak tokeny uwierzytelniające, przechowywane są w *FlutterSecureStorage*. Do wstrzykiwania zależności wykorzystano paczkę *GetIt*, pełniącą rolę mechanizmu *Service Locator* i realizującą zasadę odwrócenia sterowania (*Inversion of Control*).

3.2.4 Testy i jakość kodu

O bezbłądność architektury dba zestaw **33 automatycznych testów jednostkowych**. Spójność i jakość zapisu egzekwuje rygorystyczny zestaw reguł lintera (*flutter_lints*). Narzędzie *Freezed* zabezpiecza modele danych, narzucając im kompletną niemutowalność (*immutability*).

3.3 Aplikacja webowa (React / TypeScript)

3.3.1 Stos technologiczny

Tabela 6: Stos technologiczny aplikacji webowej

Technologia	Wersja	Zastosowanie
React	18	Biblioteka UI
TypeScript	5	Typowanie statyczne
Vite	5	Bundler i serwer deweloperski
React Router	6	Routing SPA
Axios	–	Komunikacja HTTP
React Hook Form	–	Zarządzanie formularzami
Zod	–	Walidacja schematów
Vitest	–	Testy jednostkowe

3.3.2 Architektura aplikacji

Aplikacja jest jednostronicową (SPA) z routingiem po stronie klienta. Komunikacja z backendem odbywa się przez REST API przy użyciu biblioteki Axios. Kod podzielony jest na warstwy:

- `src/api/` – warstwa komunikacji z API (`axiosConfig`, `authApi`, `herbariaApi`, `friendsApi`, `notificationsApi`),
- `src/pages/` – komponenty stron (`LoginPage`, `RegisterPage`, `Dashboard`, `PublicHerbaria`, `FriendsPage`, `PlantDetail` i inne),
- `src/components/` – komponenty wielokrotnego użytku (`ProtectedRoute`, `NotificationBell`),
- `src/hooks/` – hooki React (`useTranslation`, `useIdleLogout`, `useNotifications`),
- `src/i18n/` – pliki tłumaczeń (`pl.ts`, `en.ts`),
- `src/types/` – definicje typów TypeScript,
- `src/Utils/` – funkcje pomocnicze (`photoUrl`),
- `src/schemats/` – schematy walidacji Zod.

3.3.3 Wzorce projektowe

W projekcie świadomie zastosowano następujące wzorce projektowe:

- **Observer** – hook `useTranslation` nasłuchuje na zdarzenie `langChange` (`CustomEvent`) rozsyłane przy zmianie języka; wszystkie komponenty aktualizują się bez przeładowania strony.
- **Strategy** – funkcja `createRegisterSchema(lang)` dynamicznie tworzy schemat walidacji Zod z komunikatami w odpowiednim języku.
- **Singleton** – plik `axiosConfig.ts` eksportuje jedną instancję klienta Axios współdzieloną przez cały projekt.
- **Proxy** – interceptory Axios przechwytyją wszystkie żądania HTTP: request interceptor dokłada token JWT, response interceptor obsługuje wygaśnięcie sesji i automatyczny refresh tokenu.
- **Facade** – obiekty `herbariaApi`, `friendsApi`, `authApi` ukrywają szczegóły komunikacji HTTP za prostym interfejsem metod.
- **Null Object** – funkcja `photoUrl()` zwraca `null` zamiast rzucać wyjątek dla niebezpiecznych protokołów URL (`javascript:`, `data:`, `vbscript:`).
- **Chain of Responsibility** – interceptory Axios tworzą łańcuch przetwarzania: dodanie tokenu, wysłanie żądania, próba odświeżenia przy 401, przekierowanie do logowania.
- **Mediator** – komponent `AppContent` koordynuje routing, pasek nawigacji, idle logout i powiadomienia bez bezpośredniej komunikacji między tymi elementami.

3.3.4 Autoryzacja i internacjonalizacja

Autoryzacja opiera się na tokenach JWT. Token dostępu przechowywany jest w `localStorage` i dołączany do każdego żądania przez interceptor Axios. Przy odpowiedzi 401 interceptor automatycznie próbuje odświeżyć sesję; w razie niepowodzenia użytkownik jest wylogowywany i przekierowywany do strony logowania. Dodatkowo hook `useIdleLogout` automatycznie wylogowuje użytkownika po 15 minutach bezczynności.

System `i18n` oparty jest na własnej implementacji bez zewnętrznych bibliotek. Tłumaczenia przechowywane są w plikach `pl.ts` i `en.ts` jako obiekty TypeScript. Zmiana języka rozsyłana jest przez `CustomEvent` na obiekcie `window`, dzięki czemu wszystkie komponenty reagują na zmianę bez potrzeby przekazywania stanu przez props.

3.4 Aplikacja desktopowa (WPF / .NET)

3.4.1 Stos i architektura

Aplikacja desktopowa jest natywnym panelem administracyjnym zbudowanym w technologii **WPF (.NET 8)** przy użyciu języka **C#**. Projekt zrealizowano zgodnie z architekturą **MVVM** (Model–View–ViewModel). Stos technologiczny zestawia tabela 7.

Tabela 7: Stos technologiczny aplikacji desktopowej

Technologia	Zastosowanie
C#	Logika aplikacji
.NET 8	Platforma uruchomieniowa
WPF	Interfejs użytkownika
MVVM	Architektura aplikacji
HttpClient	Komunikacja z API
SQLite	Lokalna baza danych
JWT	Uwierzytelnianie
xUnit	Testy jednostkowe
CommunityToolkit.Mvvm	Obsługa MVVM
XAML	Definicja interfejsu

Aplikację podzielono na warstwy. **Modele** przechowują dane otrzymywane z API (m.in. `AdminUserResponse`, `PlantResponse`, `PlantDetailsResponse`, `NotificationResponse`, `StatsOverviewResponse`, `ApiResponse<T>`). **Usługi** odpowiadają za komunikację z API i operacje biznesowe (`AuthService`, `UsersService`, `PlantsService`, `NotificationsService`, `InternetService`, `LocalDatabaseService`). **ViewModele** pośredniczą między widokami a usługami (`LoginViewModel`, `MainViewModel`, `UsersViewModel`, `PlantsViewModel`, `NotificationsViewModel`). **Widoki** tworzone są przy użyciu WPF i XAML.

3.4.2 Komunikacja z API i bezpieczeństwo

Panel komunikuje się z REST API za pomocą klasy `HttpClient`. Logowanie realizowane jest poprzez API: po poprawnym uwierzytelnieniu pobierany jest token JWT, przechowywany przez `AuthService`, a wszystkie kolejne żądania są autoryzowane. Aplikacja automatycznie wylogowuje użytkownika po 5 minutach bezczynności.

W zakresie bezpieczeństwa zaimplementowano: uwierzytelnianie JWT, autoryzację administratora, automatyczne wylogowanie po bezczynności, gwarancję pojedynczej instancji aplikacji (Mutex), obsługę błędów HTTP oraz centralizację logiki autoryzacji.

3.4.3 Tryb offline i synchronizacja

Jednym z wymagań projektu była możliwość działania w trybie offline. W tym celu zaimplementowano lokalną bazę SQLite obsługiwaną przez `LocalDatabaseService` (tworzenie bazy i tabel oraz zarządzanie kolejką synchronizacji). Lokalna baza zawiera tabele: `Herbaria` (zielniki), `Plants` (rośliny), `Photos` (zdjęcia), `SyncQueue` (operacje oczekujące na synchronizację) oraz `Metadata` (metadane aplikacji).

W przypadku braku połączenia dane przechowywane są lokalnie. Po odzyskaniu połączenia odczytywana jest kolejka `SyncQueue`, operacje są wysyłane do API, a poprawnie wykonane oznaczane jako zsynchronizowane. Stan połączenia monitoruje komponent `InternetService`, sprawdzając go cyklicznie co 3 sekundy; przy utracie połączenia aplikacja przechodzi w tryb offline, korzysta z lokalnych danych i informuje użytkownika o zmianie stanu.

Aplikacja implementuje również lokalny cache zdjęć (`PhotoCacheService`): pobiera zdjęcia z API, przechowuje je lokalnie, umożliwia odczyt bez połączenia oraz automatycznie usuwa pliki starsze niż 7 dni, co ogranicza liczbę zapytań do serwera.

3.4.4 Wielojęzyczność, wzorce i testy

Aplikacja wspiera język polski i angielski. Realizację oparto na `ResourceDictionary` oraz klasie `LanguageManager` (pliki `Strings.pl.xaml` i `Strings.en.xaml`); zmiana języka odbywa się dynamicznie, bez ponownego uruchamiania aplikacji.

W projekcie zastosowano wzorce projektowe: **Singleton** (m.in. `AuthService`, `UsersService`, `InternetService`, `LocalDatabaseService`, `MainViewModel`), **MVVM** jako podstawowy wzorzec architektoniczny, **Repository** (`LocalPlantsRepository`, `LocalPhotosRepository`, `LocalHerbariaRepository`), **Command** (`RelayCommand`, `AsyncRelayCommand`) oraz **Observer** (`INotifyPropertyChanged`, `InternetStatusChanged`).

Projekt zawiera testy jednostkowe wykonane przy użyciu biblioteki `xUnit`, obejmujące między innymi `AuthService`, logikę wylogowania, przechowywanie tokenów i danych użytkownika oraz podstawowe elementy `ViewModeli`. Wszystkie testy przechodzą pomyślnie.

4 Podział obowiązków i odpowiedzialności w zespole

Projekt e-zielnik był realizowany przez czteroosobowy zespół. Przyjęto podział oparty o platformy docelowe: każdy z członków zespołu odpowiadał za odrębny moduł systemu oraz jego repozytorium. Centralnym elementem łączącym wszystkie aplikacje klienckie jest moduł REST API.

W ramach modułu serwerowego (Maksym Wilk) zrealizowano projekt architektury warstwowej i modelu danych, implementację wszystkich kontrolerów REST i logiki serwisów, kompletny podsystem uwierzytelniania (JWT, refresh tokeny z rotacją, 2FA, weryfikacja e-mail, reset hasła), integracje zewnętrzne (PlantNet, Brevo, Firebase Cloud Messaging), podsystem zielników, roślin i zdjęć, funkcje społecznościowe i powiadomienia, panel administracyjny, mechanizmy bezpieczeństwa, komplet testów jednostkowych i integracyjnych oraz konfigurację konteneryzacji, proces CI i wdrożenie zdalne.

Tabela 8: Podział odpowiedzialności członków zespołu projektowego

Członek zespołu	Nr al- bumu	Moduł i technologie
Maksym Wilk	43900	REST API (część serwerowa) – Java, Spring Boot, Maven, PostgreSQL
Adam Rudziewicz	43882	Aplikacja mobilna – Dart, Flutter
Szymon Rogula	43880	Aplikacja webowa – TypeScript, React, Vite
Sebastian Waga	43894	Aplikacja desktopowa – C#, .NET

W ramach modułu mobilnego (Adam Rudziewicz) zaprojektowano i zaimplementowano aplikację we Flutterze w architekturze Clean Architecture z BLoC, warstwę UX/UI w stylu Material Design, lokalną bazę Hive, interceptory Dio, integrację z Firebase FCM, testy jednostkowe oraz proces budowania pakietów APK.

W ramach modułu webowego (Szymon Rogula) powstała responsywna aplikacja w React i TypeScript wraz z implementacją logiki biznesowej frontendu, systemu i18n oraz integracją z API.

W ramach modułu desktopowego (Sebastian Waga) zrealizowano natywny panel administracyjny w technologii C# i .NET (WPF) z architekturą MVVM, trybem offline opartym o SQLite, synchronizacją danych oraz lokalnym cache zdjęć.

5 Instrukcja uruchomienia systemu

5.1 Moduł serwerowy (REST API)

5.1.1 Wymagania wstępne

- JDK 21 (zalecany Eclipse Temurin),
- działająca instancja PostgreSQL (dla uruchomienia lokalnego),
- opcjonalnie Docker i Docker Compose (dla uruchomienia kontenerowego),
- klucz API serwisu PlantNet; opcjonalnie klucz Brevo oraz dane konta serwisowego Firebase.

Maven nie musi być instalowany globalnie – w repozytorium znajduje się wrapper (`./mvnw`).

5.1.2 Konfiguracja zmiennych środowiskowych

Aplikacja wczytuje konfigurację z pliku `.env.properties` w katalogu głównym (import opcjonalny w `application.properties`). Przykładowa zawartość:

```
DB_URL=jdbc:postgresql://localhost:5432/
DB_USERNAME=postgres
DB_PASSWORD=postgres
JWT_SECRET=<dlugi-losowy-sekret-base64>
JWT_EXPIRATION_MS=604800000
```

```

JWT_REFRESH_EXPIRATION_DAYS=30
PLANTNET_API_KEY=<klucz-plantnet>
BREVO_API_KEY=<klucz-brevo-opcjonalnie>
MAIL_FROM=ezielnik.app@gmail.com
APP_BASE_URL=http://localhost:8080
PHOTO_STORAGE_PATH=./photos
PORT=8080

```

Listing 3: Przykładowy plik .env.properties

Plik `.env.properties` zawiera dane wrażliwe i nie jest umieszczany w repozytorium; wartości produkcyjne dostarczane są wyłącznie przez zmienne środowiskowe. Najważniejsze zmienne zestawia tabela 9.

Tabela 9: Najważniejsze zmienne środowiskowe modułu API

Zmienna	Znaczenie
DB_URL, DB_USERNAME, DB_PASSWORD	Parametry połączenia z bazą PostgreSQL
JWT_SECRET	Sekret podpisujący tokeny JWT (wymagany)
JWT_EXPIRATION_MS	Czas życia tokenu dostępu
JWT_REFRESH_EXPIRATION_DAYS	Czas życia refresh tokenu (domyślnie 30 dni)
PLANTNET_API_KEY	Klucz API serwisu identyfikacji roślin
BREVO_API_KEY	Klucz API poczty transakcyjnej (opcjonalny)
FIREBASE_SERVICE_ACCOUNT_JSON	Konto serwisowe Firebase dla powiadomień push (opcjonalne)
PHOTO_STORAGE_PATH	Katalog przechowywania zdjęć
APP_BASE_URL	Publiczny adres aplikacji (linki w wiadomościach e-mail)
PORT	Port nasłuchu serwera (domyślnie 8080)

5.1.3 Uruchomienie lokalne

1. Uruchom lokalną bazę PostgreSQL nasłuchującą na `localhost:5432` (zgodnie z `DB_URL`).
2. Utwórz plik `.env.properties` zgodnie z powyższym wzorem.
3. Schemat bazy zostanie utworzony automatycznie przy starcie aplikacji (Hibernate, tryb `update`).
4. Uruchom aplikację jednym ze sposobów:

```

1 # wariant 1: bezpośrednie uruchomienie
2 ./mvnw spring-boot:run
3
4 # wariant 2: budowa pakietu i uruchomienie artefaktu
5 ./mvnw clean package

```

```
6 java -jar target/S6-ZMP-System-identyfikacji-flory-API-1.0.jar
```

Listing 4: Uruchomienie w trybie deweloperskim

Po starcie aplikacja jest dostępna pod adresem `http://localhost:8080`, a dokumentacja Swagger UI pod `http://localhost:8080/swagger-ui.html`.

5.1.4 Uruchomienie kontenerowe (Docker Compose)

Repozytorium zawiera pliki `Dockerfile` (build wieloetapowy) oraz `compose.yaml` definiujący usługi bazy danych i aplikacji wraz z trwałym wolumenem na zdjęcia. Pełne środowisko można uruchomić jednym poleceniem:

```
1 docker compose up --build
```

Listing 5: Uruchomienie przez Docker Compose

Compose ustawia adres bazy na `jdbc:postgresql://postgres:5432/postgres`, katalog zdjęć na `/data/photos` (zamontowany jako wolumen `photos`) oraz publikuje port 8080.

5.1.5 Uruchomienie zdalne (Railway)

Aplikacja jest wdrażana na platformie Railway na podstawie pliku `Dockerfile`. Kroki wdrożenia:

1. Platforma buduje obraz na podstawie wykrytego `Dockerfile` (build wieloetapowy: kompilacja w obrazie JDK, uruchomienie w obrazie JRE).
2. Należy podłączyć usługę PostgreSQL Railway, która dostarcza parametry połączenia (`DB_URL`, `DB_USERNAME`, `DB_PASSWORD`). Port nasłuchu jest wstrzykiwany przez zmienną `PORT`.
3. W panelu Railway ustawia się zmienne środowiskowe produkcyjne: `JWT_SECRET`, `PLANTNET_API_KEY`, opcjonalnie `BREVO_API_KEY`, `FIREBASE_SERVICE_ACCOUNT_JSON`, `MAIL_FROM` oraz `APP_BASE_URL`.
4. Zdjęcia przechowywane są na trwałym wolumenie Railway (`PHOTO_STORAGE_PATH=/data/photos` z podłączonym Railway Volume zamontowanym pod `/data/photos`). Dzięki temu pliki graficzne są odizolowane od kontenera aplikacji i zachowywane między kolejnymi wdrożeniami.

5.2 Aplikacja mobilna (Flutter)

5.2.1 Uruchomienie deweloperskie

1. **Konfiguracja kluczy** – utworzenie pliku środowiskowego `.env` w katalogu głównym projektu ze zmienną wskazującą adres serwera backendowego:

```
API_BASE_URL=http://<ADRES_IP_SERWERA>:8080
```

Listing 6: Zawartość pliku `.env`

2. **Pobranie zależności** – instalacja paczek menedżerem pakietów Flutter:

```
flutter pub get
```

3. **Generowanie kodu** – przetworzenie plików generujących tłumaczenia oraz serializatory modeli:

```
dart run build_runner build -d
```

4. **Rozruch** – kompilacja w trybie *Just-In-Time* na emulatorze lub fizycznym urządzeniu:

```
flutter run
```

5.2.2 Budowa pakietu produkcyjnego (APK)

Do dystrybucji wymagane jest zbudowanie pakietu *Android Package Kit* (APK) metodą *Ahead-of-Time* (AOT):

```
flutter build apk --release
```

Gotowy plik binarny znajduje się pod ścieżką `build/app/outputs/flutter-apk/app-release.apk`. Plik należy skopiować na urządzenie z systemem Android i potwierdzić zgodę na instalację. Warunkiem korzystania ze wszystkich funkcji jest dostępny z poziomu urządzenia serwer API.

5.3 Aplikacja webowa (React)

5.3.1 Wymagania wstępne

- Node.js w wersji 18 lub nowszej,
- npm w wersji 9 lub nowszej,
- dostęp do internetu (backend działa na Railway).

5.3.2 Uruchomienie lokalne

1. Sklonuj repozytorium i przejdź do katalogu projektu:

```
git clone https://github.com/SX2V/S6-ZMP-System-identyfikacji-flory-Web
.git
cd S6-ZMP-System-identyfikacji-flory-Web
```

2. Zainstaluj zależności:

```
npm install
```

3. Uruchom serwer deweloperski:

```
npm run dev
```

4. Otwórz przeglądarkę pod adresem `http://localhost:5173`.

Serwer deweloperski Vite automatycznie konfiguruje proxy, które przekierowuje żądania z `/api/*` na backend, omijając ograniczenia CORS przeglądarki.

5.3.3 Uruchomienie produkcyjne (Vercel)

Aplikacja jest wdrożona i dostępna pod adresem:

<https://s6-zmp-system-indentyfikacji-flory-snowy.vercel.app>

Wdrożenie odbywa się automatycznie przy każdym pushu na gałąź `main` repozytorium GitHub. Vercel wykrywa projekt jako aplikację Vite i wykonuje:

```
npm run build # tsc -b && vite build
```

Wynikowy katalog `dist/` jest serwowany jako statyczna strona. Testy uruchamia się poleceniem `npm run test`.

5.4 Aplikacja desktopowa (WPF)

5.4.1 Wymagania wstępne

- system Windows,
- .NET 8 SDK,
- dostęp do działającego serwera API (lokalnego lub zdalnego).

5.4.2 Uruchomienie i budowa

Po sklonowaniu repozytorium i przejściu do katalogu projektu należy pobrać zależności i uruchomić aplikację:

```
dotnet restore
dotnet run
```

Wersję produkcyjną buduje się w konfiguracji `Release`:

```
dotnet build -c Release
```

Lokalna baza SQLite oraz katalog cache zdjęć tworzone są automatycznie przy pierwszym uruchomieniu, co umożliwia natychmiastową pracę w trybie offline.

6 Wnioski projektowe

6.1 Moduł serwerowy (Maksym Wilk)

Realizacja modułu serwerowego potwierdziła wartość klasycznej architektury warstwowej oraz konsekwentnego oddzielenia obiektów DTO od encji bazodanowych. Taki podział ułatwił rozwój projektu i ograniczył ryzyko niezamierzonego ujawnienia struktury bazy w odpowiedziach API. Organizacja kodu w pakiety dziedzinowe znacząco ułatwiła orientację w rozrastającym się repozytorium.

Przyjęty model bezpieczeństwa, oparty o bezstanowe uwierzytelnianie JWT z rotacją refresh tokenów i wykrywaniem ich ponownego użycia, okazał się zarazem bezpieczny i wygodny w utrzymaniu. Rozróżnianie typów tokenów przez deklarację `purpose` skutecznie wyeliminowało całą klasę błędów związanych z użyciem tokenu w niewłaściwym kontekście.

Szczególnie cennym doświadczeniem było oparcie testów integracyjnych o Testcontainers i realną bazę PostgreSQL. Testy uruchamiane na rzeczywistym silniku bazodanowym

dały znacznie większą pewność poprawności niż atrapy, a próg pokrycia 80 % weryfikowany w procesie CI wymusił dyscyplinę testową przez cały okres projektu. Bezpieczna degradacja integracji zewnętrznych (PlantNet, Brevo, FCM) potwierdziła, że traktowanie usług zewnętrznych jako zawodnych z założenia prowadzi do stabilniejszego systemu.

6.2 Aplikacja mobilna (Adam Rudziewicz)

Praca z frameworkiem Flutter unaocniła wartość rygorystycznego trzymania się wzorców **Clean Architecture** oraz **BLoC**. Choć początkowy próg wejścia w tak restrykcyjną abstrakcję (odseparowanie każdego interfejsu od implementacji i wiązanie ich poprzez centralny *Service Locator* – *GetIt*) wydawał się czasochłonny, długofalowo zysk był bezsporny. Ochroniło to projekt przed degradacją struktury i powstawaniem architektury typu *Spaghetti Code*.

Utrzymanie zasad **SOLID** okazało się trafną decyzją, co przejawiało się w wysokiej łatwości wdrażania testów jednostkowych. Możliwość bezinwazyjnej podmiany wstrzykniętych interfejsów (*Mocking*) pozwoliła zasymulować ciężkie błędy sieciowe i napisać 33 testy logiki. Płynne wdrożenie trybu offline z użyciem bazy Hive potwierdziło, że szczelne oddzielenie warstwy danych od interfejsu gwarantuje tolerancję na utratę połączenia (*Graceful Degradation*). Ekosystem Fluttera dowiódł przy tym wysokiej wydajności renderingu, dobrze odwzorowując wytyczne *Material Design*.

6.3 Aplikacja webowa (Szymon Rogula)

Praca nad modulem frontendowym w architekturze React/TypeScript pozwoliła sformułować istotne wnioski dotyczące projektowania nowoczesnych aplikacji typu SPA. Zastosowanie wzorca **Observer** poprzez mechanizm *CustomEvents* do obsługi internacjonalizacji okazało się trafne – pozwoliło uniknąć narzutu związanego z globalnym kontekstem Reacta przy zachowaniu pełnej reaktywności interfejsu.

Integracja **React Hook Form** z silnikiem walidacji **Zod** w środowisku wielojęzycznym unaocniła wyzwania związane z cyklem życia komponentów; konieczność manualnego odświeżania walidacji po zmianie języka doprowadziła do stworzenia stabilnego systemu obsługi formularzy. Kluczowym aspektem bezpieczeństwa było wdrożenie interceptorów **Axios**: mechanizm *Silent Refresh Token* w warstwie **Proxy** zapewnił pełną separację logiki autoryzacyjnej od warstwy prezentacji. Dyscyplina typowania wprowadzona przez **TypeScript** w trybie *strict* okazała się bezcenna podczas integracji z zewnętrznym API, a **Vite** zapewnił optymalne parametry ładowania aplikacji.

6.4 Aplikacja desktopowa (Sebastian Waga)

Realizacja panelu administracyjnego w technologii WPF potwierdziła, że konsekwentne stosowanie wzorca **MVVM** (wspartego biblioteką *CommunityToolkit.Mvvm*) prowadzi do czytelnego, łatwego w utrzymaniu i testowaniu kodu. Wyraźne oddzielenie widoków od logiki w *ViewModelach* ułatwiło rozwój aplikacji oraz pisanie testów jednostkowych dla usług.

Największym wyzwaniem, a zarazem najwartościowszym doświadczeniem, było wdrożenie pełnego trybu offline. Połączenie lokalnej bazy SQLite, kolejki synchronizacji oraz monitorowania połączenia przez **InternetService** pozwoliło płynnie przełączać aplikację między trybem online i offline bez utraty danych. Lokalny cache zdjęć z automatycznym

usuwaniem starszych plików ograniczył liczbę zapytań do serwera. Zastosowanie wzorców Repository, Singleton, Command i Observer uporządkowało dostęp do danych oraz obsługę akcji użytkownika, czyniąc projekt gotowym do dalszej rozbudowy.

7 Podsumowanie

Zrealizowany system e-zielnik stanowi kompletne, wielomodułowe rozwiązanie do identyfikacji i katalogowania flory. Centralny moduł REST API pokrywa wszystkie wymagania funkcjonalne warstwy serwerowej, a trzy aplikacje klienckie – mobilna, webowa i desktopowa – udostępniają te funkcje użytkownikom na różnych platformach.

Pod względem technicznym projekt opiera się na aktualnych technologiach (Java 21 i Spring Boot, Flutter, React/TypeScript, .NET 8) oraz sprawdzonych wzorcach projektowych. We wszystkich modułach położono nacisk na bezpieczeństwo (bezstanowe uwierzytelnianie JWT, rotacja tokenów, kontrola dostępu), jakość kodu (testy jednostkowe i integracyjne) oraz dobre praktyki inżynierii oprogramowania. Przygotowano także pełną ścieżkę uruchomienia każdego modułu: lokalną, kontenerową oraz zdalną. Przyjęty podział prac na platformy docelowe, ze wspólną warstwą integrującą w postaci REST API, pozwolił czteroosobowemu zespołowi sprawnie zrealizować spójny i kompletny system.